

L4 总结:多工具编排 + 阶段一 Capstone

阶段一(Beginner → 迈向 Intermediate)的收官课。这一节把 L2~L4 综合成一个能在 终端多轮对话的客服 agent,并在实战里踩通了“LLM 当路由器”这个模式和它的坑。

一、这节课学到的核心

1. 工具变多会崩在两个地方(还 L3 埋的债)

- ① **选择困难**:工具越多,description 路由质量越差,模型选错/漏选。经验阈值:单次 暴露给模型的工具 > ~10 个,准确率明显下降。
- ② **上下文膨胀**:每轮请求都要塞【全部】工具 schema,几十个工具几千 token,每轮重复烧, 还挤占对话历史。→ 直连 L6 上下文管理(学习者 L2 就自己推导过“messages 越滚越长”)。

2. 本节最重要的新抽象:编排的「决策权」放哪?

L3 说“能力来自工具,不来自 loop” ;L4 补一句:“**可靠性来自约束**” 。

	模型编排(model-driven)	代码编排 / 路由(code-driven)
谁决定调哪个工具	模型在全集里自己挑	代码先按意图筛,只给相关子集
就是	L2 / L3 的做法	L4 的新东西
适合	工具少、功能不重叠	工具多、要可靠可控、要省 token

代码编排 ≠ 抢模型的活,是给它**减负**:先判断意图落哪组,本轮只暴露那组工具,模型仍在 子集里自由链式编排。

3. 进阶谱系(埋 L7 伏笔)

全放养 → **静态分组**(L4 主战场)→ **动态检索 tool RAG**(工具几百个时,把工具描述做成 向量库按 query 检索 top-k)。第三档正是学习者 L3 结尾自己想到的“按需检索工具” ,思路同 L7 RAG。

4. 一个自己悟出的分组难点(quiz Q5)

学习者做分组时自造了一个“矛盾”：查订单/查物流 既想单独成组、又要陪 改地址 成组。这不是 错,恰恰是分组的真实难点——**分组不必是硬划分**：- **硬划分**:每个工具唯一归属一组,路由简单,但通用工具很尴尬。- **共享底座 + 场景增量**:把 `query_order` 这类通用查询设成每组都带的底座,再按场景加专用 工具。生产更常用(capstone 骨架就是这么分的)。

另:路由的分组**依据**应偏“用户来干什么”(意图/场景),而非“功能是否重复”——因为路由发生在 用户开口那一刻,手里只有 `query`。

二、Capstone:命令行多工具客服 agent

把三节课综合:L2 loop + L3(多工具/错误自愈/循环上限)+ L4(分组路由)。跑通标准全过:

```
你> 查一下订单B456          → order 组,查到已签收
你> 那帮我把它退了,商品破损  → refund 组,记得"它"=B456,链式 查→退→成功
你> 我有多少积分            → account 组,3200 分
你> 退出
```

三个亲手补的关键点: 1. **分组路由**(TODO 1/2): `route()` 判断意图 → 取对应工具组 → 只把这组传给模型。 2. **会话内多轮记忆**(TODO 3):维护一份贯穿会话的 `messages`,循环 `append`,agent 天然记得 上文 (“它”指哪个订单)。这已摸到 **L5 记忆的门槛——是学习者自己搭出来的**。 3. 采纳更干净的架构:把 `user_text` 传进 `run_agent`,不去混血 `messages` 列表里“考古”。

三、实战踩的坑(本节最值钱的部分)

坑 1: ‘dict’ object has no attribute ‘role’

`messages` 是**混血列表**:自己手写的 `system/user/tool` 是 `dict` (用 `m["role"]`),API 回来 `append` 的 `assistant` 是 SDK 对象(用 `m.role`)。用统一方式访问必炸一头。 > 教训:一个 `list` 里混着 `dict` 和 SDK 对象,是这类 `AttributeError` 的常见来源。

坑 2:LLM 路由“静默失灵”,三句全走 `order`

换成 LLM 路由后,所有输入都被兜底成 `order`。加一行 `debug print` 打出模型原始返回:

```
[router] 原始='' strip后='' ← 模型返回空字符串!
```

根因: deepseek-v4-flash 是**思考型模型**,输出前先烧 reasoning token; max_tokens=5 在它思考完之前就耗光预算 → content 一个字没剩 → 空串 → else "order" 一直兜底。

修法:路由改用**非思考的快模型** deepseek-chat (给对的活挑对的模型)。换完 [router] 原始='refund' 立刻正常。

两条硬道理: 1. **fallback 是双刃剑**——让程序不崩,也把 bug 藏起来。写兜底的地方,调试期一定要能看见“兜底之前的真实值”(用 repr / !r 暴露隐藏字符)。2. **“错误也是信息”的升级版**:L3 是工具报错也是信息;这次是 **LLM 的“沉默”(空返回)也是信息**。路由/分类要的是快和便宜,别用会写小作文的推理模型。

四、LLM-as-router 模式的 4 个坑(备查)

1. **约束输出**:system prompt 死钉“只输出类别名,不要解释”,压住模型话痨天性。
2. **temperature=0**:分类要确定性,别让它飘。
3. **max_tokens** 别太小(16~64),尤其防思考型模型把预算烧在推理上。
4. **必须校验 + 兜底**: label if label in VALID_GROUPS else 默认组,防它返回中文/带标点/整句话。

五、Quiz 回顾(概念 5/5 全过)

- Q1 工具多了“多写几个”哪对哪不对 → 对在 loop 不改,崩在选择困难 + schema 吃 token。✅
- Q2 模型编排 vs 代码编排的区别与适用场景。✅
- Q3 代码编排多做了“意图→工具组”这步,减掉模型的选择负担。✅
- Q4 恶化的是上下文/token(L6),学习者 L2 自己就推导过。✅
- Q5 静态分组设计 → 悟出“共享工具可跨组”,即共享底座模式。★

六、埋点(后续课回扣)

- **L5 记忆**:capstone 的多轮 messages 就是会话内记忆;L5 讲它的边界(越滚越长、跨会话)。
- **L6 上下文管理**:schema 吃 token、messages 膨胀,都在这里正式收。
- **L7 RAG**:tool RAG(按需检索工具)= 学习者自己想到的思路。
- **L10 安全**:修改密码/注销这类高危操作,光分组不够,要加二次确认/权限校验。
- **延迟调优**:已知主模型 deepseek-v4-flash 是思考型,每次主调用都在偷偷思考(多花延迟)。